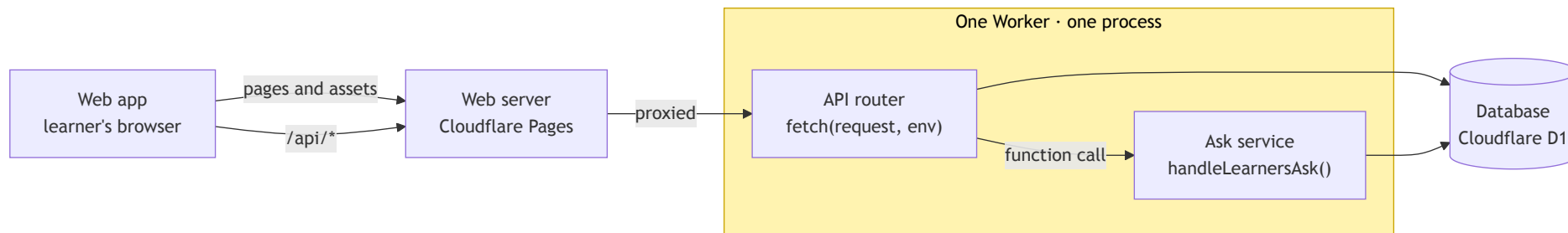


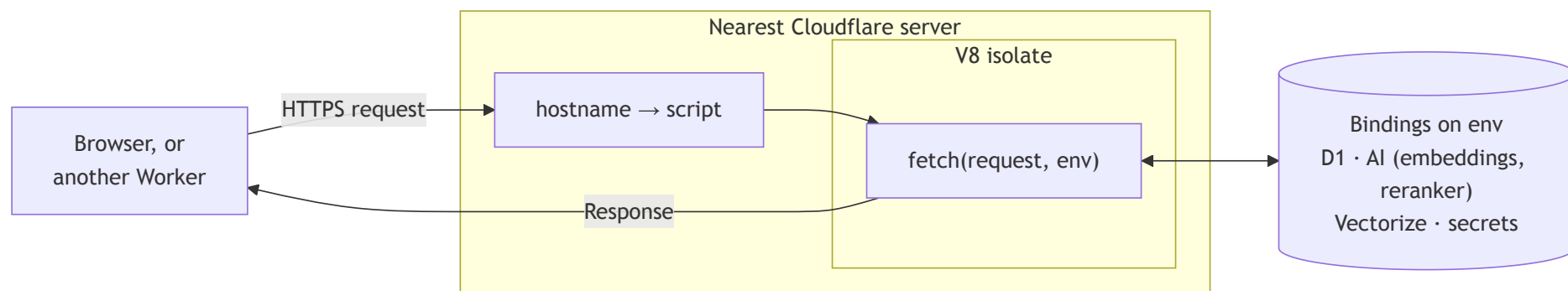
Architecture at a glance



Yellow: code running in the same process.

Cloudflare Workers in brief

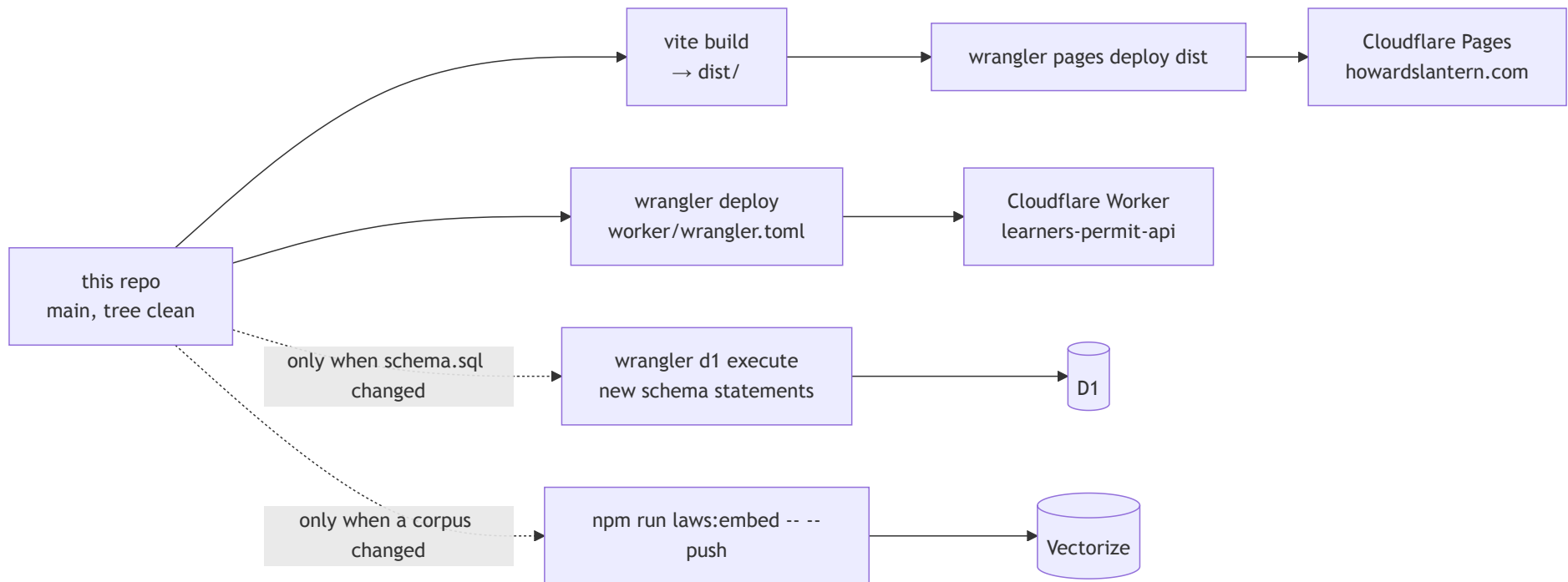
A Worker is a script, not a server. You upload one JavaScript file; Cloudflare runs it in every one of its data centres and calls it once per request.



- **Isolate, not process.** The script runs in a V8 isolate, the JavaScript engine from Chrome, without Node. Cloudflare packs many isolates into one process per server and starts a new one in under a millisecond, so there is no port to open and no cold start to speak of.
- **One call per request.** Cloudflare terminates HTTPS, finds the script by hostname, and calls its `export default { fetch }` with the request. `fetch` is an `async` function returning `Promise<Response>`; Cloudflare awaits it and sends whatever it resolves to. There is no callback and no `res.send()`: the `Response` object is the return value. Nothing survives between calls except what `env` reaches.

- **Bindings are dependency injection.** `env` is the injected dependencies: one ready-made client for each resource declared in `wrangler.toml` (D1, Workers AI, Vectorize, R2) plus the secrets. The Worker never constructs a connection or reads a connection string; Cloudflare builds the clients and passes them in on every call, and the code just uses `env.DB`.
- **Billed on CPU time, not wall time.** Waiting on the network, such as for Claude's reply, costs nothing.
- **Pages Functions are Workers too**, run in front of a static site. The proxy in this app is one.
- **Turnstile** is Cloudflare's CAPTCHA replacement: the browser gets a one-time token from a challenge the visitor rarely sees, and the Worker redeems it with Cloudflare over HTTPS to confirm a person, not a script, sent the request.

Deploying



- **Pages:** `npm run deploy` builds `dist/` from the working tree and uploads it; the app, the prerendered pages and the per-state laws files all ship in that one upload.
- **Worker:** `npx wrangler deploy --config worker/wrangler.toml` bundles `worker/src/index.ts` and uploads it under the name in that file; the bindings in the file are registered with it.
- **D1:** has no deploy; new statements from `worker/schema.sql` are applied by hand to the production database before the Worker that reads them goes out.

- **Vectorize:** filled by hand with `npm run laws:embed -- --push` after a corpus change, which embeds every law and upserts it.
- **Secrets** (`ANTHROPIC_API_KEY`, `STRIPE_SECRET_KEY`, ...) are set once with `wrangler secret put` and survive every deploy.

How the boxes talk

- **Web app → Web server:** HTTPS to `howardslantern.com`. Pages serves the files; `/api/*` is handed to a small function that forwards it.
- **Web server → API ("proxied"):** that function makes an ordinary HTTPS request to `learners-permit-api.joelstevick.workers.dev` and returns the reply, so the browser only ever sees one site.
- **The Worker's address** is that hostname. Cloudflare assigned it at deploy and it points at Cloudflare's own servers, which listen on port 443 for every hostname they host. When a request for it arrives, Cloudflare looks up which Worker is deployed under that name and calls the Worker's one function, `fetch(request, env)`, with the request. Nothing of ours listens on any port; the address is a name in Cloudflare's routing table, not a machine. `howardslantern.com` works the same way for Pages: two deployments, two hostnames, which is why the proxy exists.
- **/api/ask is a URL path**, `https://howardslantern.com/api/ask`. `/api/` is just the prefix that tells Pages to forward the request instead of serving a file. There is no separate address for the ask service.
- **API → Ask service:** the router reads the path off the request, and when it is `/api/ask` it calls `handleLearnersAsk`: a plain function call in the same process.
- **API / Ask service → Database:** through `env.DB`, an object Cloudflare provides, not a connection string.

How a hostname reaches the function

1. **Deploy.** `npx wrangler deploy --config worker/wrangler.toml bundles src/index.ts` into one script and uploads it under the name in that file, `learners-permit-api`. Cloudflare registers `learners-permit-api.joelstevick.workers.dev` → that script, and records the bindings (`DB`, `AI`, `LAWS`, ...) and secrets alongside it.
2. **Request.** A request arrives at the nearest Cloudflare server with that hostname in its `Host` header. The server looks the name up, finds the script, and loads it into an isolate if one is not already warm there.
3. **Call.** It builds `env` from the recorded bindings and calls the script's `export default { fetch }`. That export is the entry point Cloudflare looks for, the way a C program is entered at `main()`.
4. **Reply.** Whatever `fetch` returns is sent back as the HTTP response.

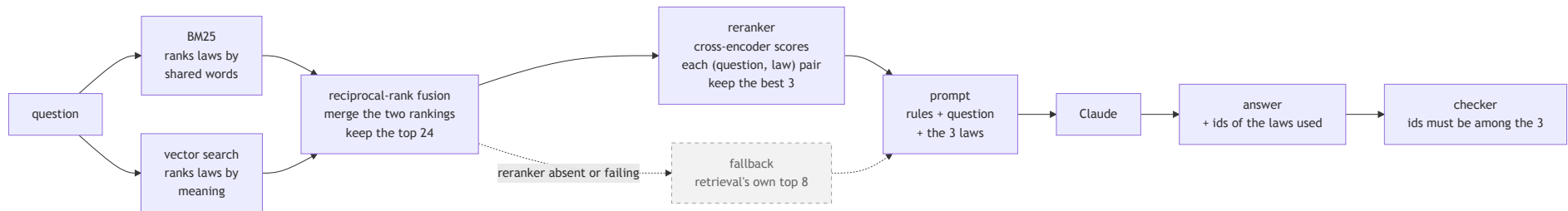
The diagram illustrates the system architecture, showing the flow from the user's browser to the database and back.

- User Interaction:** A **Web app learner's browser** sends a `/api/ask` request to a **Web server Cloudflare Pages**.
- Request Handling:** The **Web server** sends a **proxied** request to an **API router** (labeled `fetch(request, env)`).
- Worker Execution:** The **API router** triggers a **function call** to a **Gate** component within a **One Worker - one process** environment (labeled `Ask service · handleLearnersAsk()`).
- Search and Retrieval:** The **Gate** component interacts with **Retrieval**, which then branches into **BM25 keyword search** and **Model** (for **Workers AI + Vectorize vector search · reranker**).
- Processing and Output:** The **Retrieval** component also interacts with a **Reranker cross-encoder**. The **Model** component interacts with a **Checker**, which then sends data to a **Log** component.
- Data Sources and External Services:** The **Log** component interacts with a **Database Cloudflare D1**. The **Gate** component interacts with **Turnstile** and **Anthropic**. The **Workers AI + Vectorize** component interacts with **Anthropic**.
- Feedback Loop:** A **laws file** is sent from the **Web server** to the **Retrieval** component.

- **Gate** scrubs the question, checks the turn and daily limits and the human test, and claims a row in D1.
- **Retrieval** fetches the state's laws file, runs both searches below, merges their rankings into a pool of 24 candidates, and has the reranker pick the 3 the model reads.
- **BM25** is the keyword search: a plain function over the laws array in memory, scoring each law by how many of the question's words it uses and how rare those words are.
- **Vector search** is the meaning search: Workers AI embeds the question and Vectorize returns the laws whose embeddings sit nearest it. A piece of a state's vehicle code is embedded with the heading of the section it was cut from, so a bare "(c) ... shall pay \$500" still reads as an insurance law.
- **Reranker** is a cross-encoder, `bge-reranker-base` on Workers AI. It reads the question and each candidate together and scores how well that law answers it, then keeps the best 3.
- **Model** sends the question and those 3 laws to Claude and gets back an answer with the ids of the laws it used.
- **Checker** refuses an answer that cites a law it was not given or states a figure the law does not, replacing it with a one-line reply.

- **Log** records what was shown, or why not, with no ids.

How the searches, the reranker and the model fit together



The two searches cover each other's blind spot. BM25 finds a law only if the question shares its words, so it misses "fireplug" for the hydrant law and misses Spanish altogether, but it is exact when the words do match. Vector search finds laws that mean the same thing in different words, but it has no sense of exactness and will return the nearest law even when nothing is near.

Their scores are on different scales, so they are merged by position, not score: each law earns a share from wherever it ranks on each list, and the sums decide the 24 candidates. A law near the top of either list gets through; a law high on both leads.

Both searches rank by wording, and that is not the same as answering. A manual sentence written beside a question outranks the code's text for a plain question whether or not it answers it. So the pool is widened to 24 and a cross-encoder reranks it: unlike the two searches, which score the question and each law separately, it reads the pair together and scores the fit. The model gets the top 3, with two guards:

- **Retrieval's first law keeps a place.** If the reranker drops the law fusion ranked first, it takes the third slot.
- **The manual stays in.** If the 3 hold no manual law and the pool does, the manual law the reranker scored highest takes the third slot.

There is no score floor: a Spanish question scores far lower against an English law than the English question does, yet still ranks it first. If the reranker is absent or fails, the model gets what retrieval gave before the reranker existed: its own top 8, retrieved as they were.

In RAG terms: this is hybrid retrieval, a sparse retriever (BM25) and a dense retriever (bge-m3 embeddings in Vectorize) run as two first-stage candidate generators, fused with RRF, then reranked by a cross-encoder (bge-reranker-base) down to 3. The dense side is one call: `topK: 8` in the state's namespace, anything under 0.5 cosine dropped. No query rewriting or HyDE; the question is embedded as asked, after scrubbing. The checker after generation is a citation-grounding check, not part of retrieval.

The model never searches. It is handed those 3 laws as text, each piece of the vehicle code labelled with its section's heading, and told to answer from them and say which it used. The checker's job is then simple: the ids it names must be among the laws it was given, or the answer is not shown.

